TriageGuard: Privacy-First Insurance Document Triage

Milestone 1: Problem Definition and Business Case NLP Project, Industry Track | Prepared by: [Your name] | Presented to: Project Jury

1. Summary in plain words

Health insurance companies receive thousands of papers every day: claim forms, hospital bills, discharge summaries, lab reports and more. Today, a person opens each paper, decides what it is, finds the important facts, and sends it to the right team. This is slow, uneven, and it shows private patient data to everyone who touches the file.

TriageGuard is a software system that does this first sorting step automatically and safely. It:

1. Hides private identifiers (names, SSNs, phone numbers, addresses).
2. Finds out what kind of document it is.
3. Pulls out the important facts (diagnosis, dates, amounts).
4. Suggests which team should handle it: routine, high-value, or suspicious.
5. Says how sure it is. When it is not sure, a human decides.

This document is Milestone 1, so no model is built yet. Here we define the problem, the users, the goal, how we will measure success, where the data will come from, and what could go wrong. We decide what "good" means before we build anything, so we cannot move the goalposts later.

2. The problem

2.1 A day in the mailroom

Imagine the mailroom of a health insurer or a third-party administrator (TPA). By 10 a.m., 500 documents have arrived. A clerk opens the first one. It is a hospital bill. The clerk finds the patient, the diagnosis, the dates and the total amount, then decides: routine, or should a senior team see it? The clerk repeats this 500 times. By afternoon the clerk is tired, one expensive claim gets filed as routine, and a patient's full details have been seen by five different people.

2.2 Why the current way hurts

Pain	What happens	Who feels it
Slow	Every document is read by a person, so queues grow	Customers waiting, operations manager
Inconsistent	Two clerks may route the same document differently	Claims team, auditors
Private data exposed	Every handler sees full personal details; more people means more chance of a leak	Patients, privacy officer
Costly mistakes	A high-value or suspicious claim sent to the wrong queue can cause delay or money loss	Insurer, fraud team
No clear record	It is hard to explain why a document went to a certain queue	Auditors, managers

2.3 Problem statement

Insurers and TPAs receive large volumes of claim-related documents that are full of personal data. Manual triage is slow, inconsistent, and exposes private information to many people. We will build an NLP pipeline that hides personal identifiers, identifies the document type, extracts key fields, and recommends a route with a confidence score, with human review for uncertain cases.

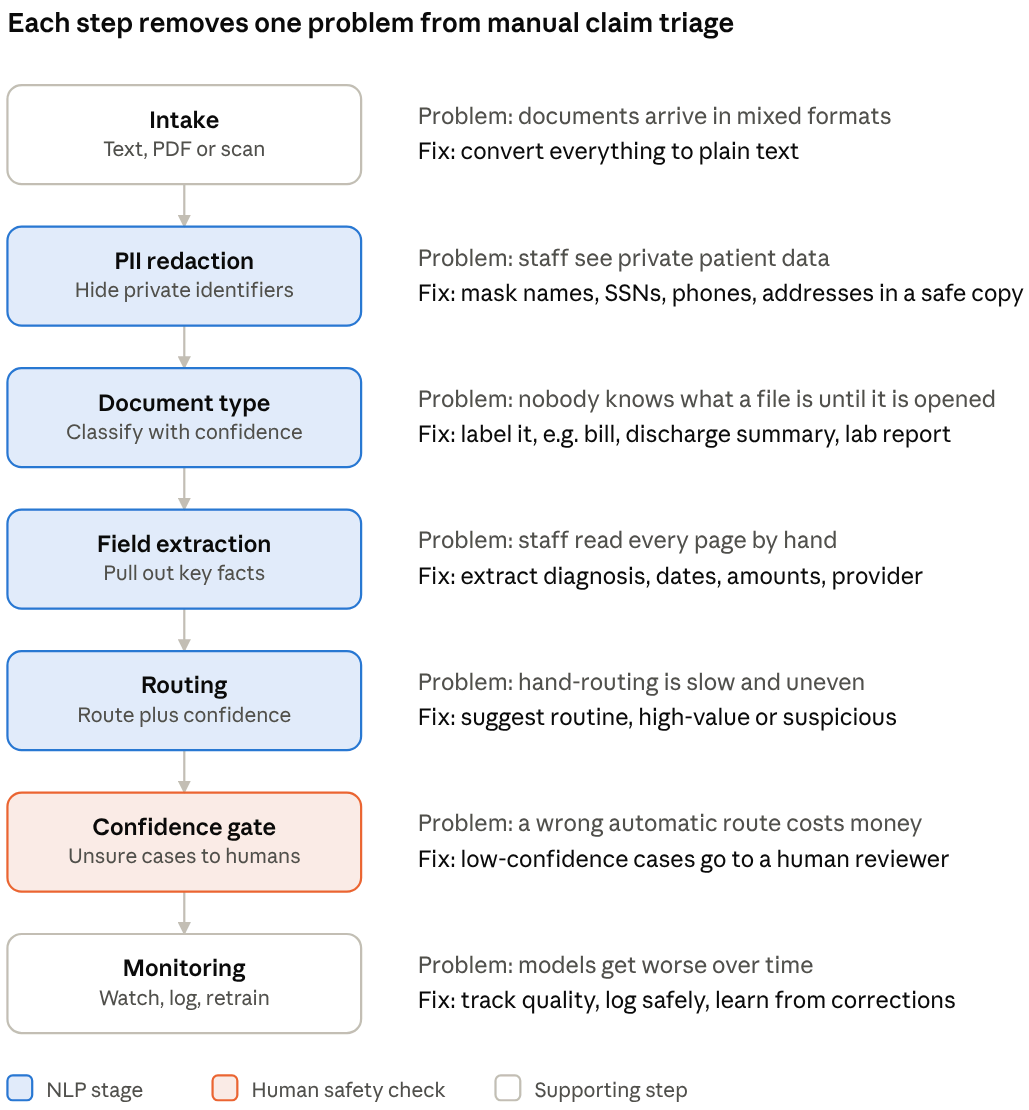
3. Our solution at a glance

Every step exists to remove one specific problem.

Step	Problem it solves	What it does	Output
Intake	Documents arrive in mixed formats	Converts text, PDF or scan into plain text	Clean text
PII redaction	Staff see private patient data	Finds and masks personal identifiers	A safe copy of the text

Step	Problem it solves	What it does	Output
Document type	We do not know what we are looking at	Labels the document (bill, summary, and so on)	Type + confidence
Field extraction	Staff read every page by hand	Pulls out diagnosis, dates, amounts, provider	Structured fields
Routing	Hand-routing is slow and uneven	Suggests routine, high-value or suspicious	Route + confidence
Confidence gate	A wrong automatic route costs money	Sends unsure cases to a human reviewer	Auto-route or review
Monitoring	Models get worse over time	Tracks quality, logs safely, learns from corrections	Health reports, new training data

(The flow diagram of these seven steps is shown below the table.)



claim triage flow · 7 steps, problem and fix at each

Read it top to bottom: each box is a step, and the text beside it says the problem it removes and how.

3.1 A worked example (all data below is made up)

Input document (text):

```
Patient: Jane Rivera   DOB: 04/12/1978   SSN: 900-12-3456
Phone: (555) 201-8890   Member ID: MB-4471920
Diagnosis: Type 2 diabetes with kidney complications
Admitted: 03/02/2026   Discharged: 03/07/2026
Total billed: $18,450.00
```

What TriageGuard returns:

```
safe_text:      Patient: [NAME]  DOB: [DATE_OF_BIRTH]  SSN: [SSN]
                Phone: [PHONE]  Member ID: [MEMBER_ID] ...
doc_type:       hospital bill (confidence 0.97)
fields:         diagnosis = type 2 diabetes with kidney complications
                admission = 2026-03-02, discharge = 2026-03-07
                total = 18450.00
route:          high-value (confidence 0.91)
needs_review:   no
```

The safe copy can be stored, logged and shown to staff who do not need the real identifiers. Only the pipeline, inside a protected area, uses the original text.

4. How each step works

For every step we first build a **simple baseline**, and then a stronger model. A fancy model is only justified if it beats the simple one.

4.1 Intake

The NLP pipeline works on text. If a file is already digital text, we use it directly. If it is a PDF with a text layer, we read the text out. If it is a scan or photo, an OCR tool (we plan to use the free Tesseract engine) converts it to text first. Section 9 explains this decision in detail.

4.2 PII redaction

PII means personally identifiable information: details that point to a real person.

- **What we hide:** names (patient, relatives, doctors), addresses (street, city, ZIP), date of birth, phone number, email, SSN, health plan member ID, medical record number, account number.
- **How:** a rule-based baseline (patterns for SSNs, phones, emails, plus word lists) first. Then a trained model that reads the sentence and marks each word as private or not. This is called token-level named entity recognition (NER). Rules are good at fixed patterns, and the model is good at names and context.
- **Output:** the text with typed placeholders, such as [NAME] or [PHONE].
- **How we judge it:** recall first, because a missed identifier is a leak.

4.3 Document type classification

Six types: claim form, hospital bill, discharge summary, lab report, prescription, pre-authorization letter. The type tells us which fields to look for. Baseline: word counts (TF-IDF) with a simple classifier. Stronger model: a small pre-trained language model fine-tuned on our data. The output includes a confidence score.

4.4 Field extraction

Document type	Key fields
Claim form	Claimed amount, diagnosis, procedure, dates, provider
Hospital bill	Total billed, itemized charges, admission and discharge dates
Discharge summary	Diagnosis, procedures, admission and discharge dates, doctor
Lab report	Tests, results, collection date
Prescription	Medicines, doses, prescribing doctor, date
Pre-authorization letter	Requested procedure, diagnosis, estimated cost

Baseline: rules and patterns. Stronger model: a trained tagger that marks each word as part of a field. We measure whether each field is found exactly right.

4.5 Routing

Three routes: **routine**, **high-value** (large amounts), and **suspicious** (patterns that deserve a closer look). We call it "suspicious" and not "fraud" on purpose. We have no real fraud labels, and we should not pretend to detect fraud. In our synthetic data, suspicious means documents with injected warning signs: duplicate billing, impossible dates, or a procedure that does not match the diagnosis.

An honest note: on synthetic data the routing labels come from rules, so a model could learn them easily. To keep the step meaningful we make routing depend on several signals together and add ambiguous cases. The real value of this step is the full pipeline, the confidence score, and the review loop. With a real insurer's data, human decisions would replace our rules as labels.

4.6 Confidence gate and human review

Every stage gives a confidence score. If the document type, the extracted fields, or the route is not confident enough, the document goes to a human reviewer. We choose the threshold by testing, balancing how many documents are automated against how many errors slip through. Confidence scores must also be honest, which we check with a calibration test.

4.7 Monitoring and learning

We track speed, error rates, how confident the model is over time, and how often humans change its answer. Reviewer corrections become new training examples. Logs never contain raw private data.

5. Who uses it

User	What they need	Which error hurts most
Intake clerk	Correct document type and fields, fast	Wrong labels that waste time
High-value claims team	Big claims escalated reliably	A missed high-value claim
Fraud analyst	Suspicious claims flagged	A missed suspicious claim (false alarms are tolerable)
Privacy officer	No raw identifiers leaving the secure area	Any leaked identifier
Operations manager (the buyer)	Higher throughput, lower cost, an audit trail	Slow or unreliable automation

6. Business objective and KPIs

Objective: Reduce manual first-pass handling of claim documents by automatically sorting most of them accurately, while never letting raw personal identifiers leave the protected area, and sending uncertain cases to people.

KPI	Meaning	Target
Auto-triage rate	Share of documents handled without a human at high accuracy	At least 70% (assumption, settled in M4)
Misroute rate	High-value or suspicious claims sent to the wrong queue	Very low; see recall targets in Section 7
PII leakage	Raw identifiers found in the safe copy	Zero tolerance goal; measured as recall
Time per document	Time to triage one document	Under 2 seconds for the software; humans only for unsure cases

Illustrative impact (an assumption, not a measurement): if a clerk spends about 3 minutes per document and 1,000 documents arrive each day, that is about 50 working hours daily. If 70% are handled automatically, about 35 of those hours are saved. We will replace the 3-minute guess with a sourced figure in the final report.

7. Success metrics and why we chose these numbers

7.1 The metrics in simple words

- **Recall:** of all the real private items, how many did we find? Missing one is a leak.
- **Precision:** of the items we flagged, how many were truly private?
- **F1:** one number that combines precision and recall.
- **Macro-F1:** the average F1 across classes, so small classes count as much as big ones.
- **Calibration error:** when the model says "90% sure", is it right about 90% of the time? Smaller is better.
- **p95 latency:** the time within which 95 out of 100 documents finish.

7.2 Targets and the reasons behind them

These targets are our **starting hypotheses**, based on three things: the cost of each kind of mistake, normal score ranges for similar tasks, and what our own baseline will show in M3. They are not copied from a rulebook.

Stage	Metric	Starting target	Why this number
PII redaction	Entity-level recall	At least 0.97	A missed identifier is a privacy leak, so this is our strictest target. Published de-identification systems report scores in the high 90s on standard benchmarks (we will cite exact papers in M2).
Document type	Macro-F1	At least 0.95	Six document types that look quite different; this is a floor, not an ambition. Macro-F1 stops big classes hiding weak small ones.
Field extraction	Field-level F1	At least 0.90	Dates, amounts and diagnoses vary more than document types, so this is harder. 0.90 means roughly 9 of 10 fields are exactly right.
Routing	Recall on high-value and suspicious	At least 0.90	Missing these costs the most. We accept extra false alarms because a human can clear them.
Confidence	Calibration error	At most 0.05	A common rule of thumb for well-calibrated models. It means "90% sure" really behaves like 85 to 95%.
System	p95 latency per document	Under 2 seconds on a CPU	Keeps hosting cheap (no GPU) and fits a mailroom workflow. This is an assumption we will test.
Business	Auto-triage rate at high accuracy	At least 70%	An assumption. In M4 we plot automation against error rate for different confidence thresholds and choose the best point.

7.3 How we keep the targets honest

1. **Two test sets.** One "golden" test set that we never tune on, and one harder set made of held-out layouts, noisy text and OCR text. Because synthetic data flatters models, we report both. On the harder set we accept up to 5 points lower and report the gap openly.
2. **Baseline first.** In M3 we build simple baselines. Every stronger model must beat its baseline on the golden set, or it does not ship.
3. **Changes in writing.** If a target proves too easy or impossible, we change it with a written reason. We never lower a target silently.

8. Data plan

8.1 What data we need

Documents in six types, containing realistic private identifiers, with correct labels for: where each identifier is, what type the document is, what the key fields are, and which route is right.

8.2 Why not an open real dataset?

Source	Why it does not fit alone
Real insurance claims	Private and protected by law. Not available to students.
MIMIC (hospital notes)	Needs credentialed access and training; notes are already de-identified (no real PII to learn from); not claim documents; licence limits sharing.
n2c2 / i2b2 de-identification sets	Need a data use agreement and waiting time; not claim documents; no routing labels.
MTSamples (public medical transcriptions)	Free to read, but not claims, no PII, no labels, and its licence is unclear. Good for varied clinical wording only.

None of these gives us claim-style documents with PII, document types, fields and routes all labelled. So we build our own synthetic corpus. Companies also use synthetic data when real data is restricted.

8.3 How we build the synthetic data

1. **Generate fake patients.** Synthea (an open-source generator) creates synthetic patients with diagnoses, procedures, visit dates, providers and claim amounts. It also supplies names, addresses and SSNs.
2. **Fill the gaps.** Faker adds phone numbers, emails, member IDs and record numbers.
3. **Write templates.** For each of the six document types we write three to four layouts and wordings, such as different bill formats or discharge summary styles.
4. **Fill templates with facts.** Each document is created from one patient's facts. Because we insert every identifier ourselves, we know exactly where it sits, so PII labels are exact.
5. **Add labels.** Document type is known from the template. Field values are known from the facts. Routing labels come from rules over several signals (amount, mismatched procedure, impossible dates, duplicate billing).
6. **Add variety.** Varied clinical wording for narrative sections, and optional re-wording of templates so the model does not memorize a few fixed phrases.
7. **Add noise.** Typos, abbreviations, odd line breaks, missing fields.
8. **Make scanned versions.** Render a share of documents to PDF and image, then run OCR to get real scan-style errors (see Section 9).
9. **Split safely.** See below.

Planned size: roughly 5,000 to 8,000 documents, spread across the six types. This is a plan, to be confirmed in M2.

8.4 Splits and leakage control

- **Split by patient.** No patient appears in more than one of train, validation and test.
- **Hold out layouts.** One layout per document type is kept out of training entirely and used only in the hard test set. This tests whether the model understands documents or just memorizes templates.
- **Frozen golden test set.** Never used for tuning.

8.5 Honest limits

- Synthetic data is easier than real data, so scores may look better than real-world performance. We counter this with held-out layouts, noise, OCR text, and by saying it plainly in the report.
- We will run a small sanity check on real public clinical text to see whether the models behave sensibly.
- Real use would need testing on a real insurer's data, under a proper agreement.

8.6 Licences

Synthea and Faker are open-source with permissive licences. We will confirm exact terms and record every source in a **data card** in M2. Using clean licences also means we own what we build.

9. Why text input, and what about scanned documents?

It is true that many real claim documents are scans or photos. Our decision is:

The core NLP system works on text. Scans are handled by an OCR step before it, and we test it.

- **This is an NLP project.** The skills graded here are understanding text: finding entities, classifying, and extracting. Reading pixels (OCR) is a separate field with its own methods and measures.
- **This is how real systems are built.** OCR is normally a separate upstream service. The NLP pipeline starts from the text it produces. Our design matches that, so OCR can be swapped later.
- **Scope.** Building a good OCR solution as well would multiply the work and risk for a solo project.

But we do not ignore scans:

1. We generate our documents as text first.
2. We render a share of them as PDF and image files.
3. We run the free Tesseract OCR tool on those images. This gives *real* OCR errors, such as "Diagnosis" becoming "Diagn0sis", or amounts like "1,500" becoming "l,5OO".
4. We use that OCR text as a robustness test, and measure how much each stage's score drops.

5. A small intake adapter reads text-layer PDFs directly and sends images to OCR, so the system accepts scans end to end.

10. Privacy and ethics

- **No real patient data is used anywhere.** Everything is synthetic.
- **HIPAA as design guidance.** HIPAA's de-identification rule lists 18 types of identifiers: names, small-area geography, dates tied to a person, phone, fax, email, SSN, medical record numbers, health plan numbers, account numbers, license numbers, vehicle and device IDs, web addresses, IP addresses, biometric identifiers, full-face photos, and other unique codes. We use it to define what "private" means. We do not claim HIPAA compliance, because that needs a formal process.
- **Two-tier handling.** *Tier 1, direct identifiers* (names, SSN, phone, email, address, member ID, record number, account number, date of birth): always masked in the safe copy. *Tier 2, working facts* (service dates, amounts, codes): needed to extract fields, so they are used only inside the protected area, and generalized in anything that leaves it.
- **Safe logging.** Logs, stored outputs and reviewer screens use the safe copy. Raw identifiers are never logged.
- **Configurable identifier list.** Our design is US-style, but the list of identifiers is a setting, so an India pack (Aadhaar, PAN) can be added later.

11. Scope

In scope	Out of scope
Redaction, document type, field extraction, routing, confidence, human review	Approving or denying claims
Six document types in English text	Medical coding and billing rules
Synthetic data, with scanned test versions	Building our own OCR engine
A working API and a live demo	Real fraud detection

Must-have: the first three stages (redaction, type, fields) plus a simple routing model and the confidence gate. **Stretch goals:** mapping diagnoses to standard medical codes (ICD-10), and a small reviewer screen.

12. Feasibility

Area	Assessment
Data	Feasible through synthetic generation. This is the biggest risk, so Section 8 details it.
Skills and tools	Python, spaCy, Microsoft Presidio and regular expressions for baselines; Hugging Face Transformers for a BERT-style model; scikit-learn; FastAPI and Docker for the app; Git and MLflow for tracking.
Compute	Free Colab or Kaggle GPUs for training; a CPU is enough for running.
Legal and ethical	Safe, because no real data is used.

Go / no-go decision: GO. The data risk is real but controlled by the synthetic plan. **Plan B** if data work runs long: drop the stretch goals, reduce to four layouts total, and use rule-based routing with a model only for confidence.

13. Risks and how we handle them

Risk	Why it matters	How we handle it
Synthetic data is too easy	Scores look better than reality	Held-out layouts, noise, OCR text, honest reporting
Models memorize templates	Fake high scores	Layout-level held-out test set
Weak routing labels	Routing is rule-based	Combine several signals, add ambiguous cases, state the limit
OCR errors hurt accuracy	Scans are messy	Test with real OCR output; improve in M4
Missed identifiers	Privacy leak	Rules plus model, recall-first target, review of misses

Risk	Why it matters	How we handle it
Scope creep	A solo project can sprawl	Must-have vs stretch list
Licence problems	Cannot share or sell	Use permissive sources, record them in the data card
Time runs short	Missed milestones	Plan B above

14. How it would run in real life (production thinking)

- **Hosting:** a web service (FastAPI) in a container (Docker), with a health check, deployable on a cloud platform or a free host for the demo.
- **Model storage:** a model registry where each version is saved with its weights, settings, data version and scores, marked as staging, production or archived. The app loads "the production version" by name.
- **Safe upgrades:** a new model (the challenger) replaces the current one (the champion) only if it wins on the golden test set, on slices, and on speed. Otherwise the current model stays and we record the negative result.
- **Fallbacks:** (1) low confidence goes to a human; (2) if a model fails or is too slow, fall back to the baseline or rules; (3) roll back by pointing to the previous registry version.

15. Decision log

Decision	Reason
Industry Track	We want an end-to-end system, not a paper reproduction.
US-style documents (HIPAA)	More public tooling and standards; configurable so India can be added.
Synthetic corpus	No public claim data exists; synthetic gives exact labels and clean licences.
Text-first with OCR tested	NLP focus; matches how real systems separate OCR; scans still tested.

Decision	Reason
Two-tier PII	Extraction needs some dates and amounts; safe copy still protects identifiers.
"Suspicious" instead of "fraud"	We have no real fraud labels.
Baseline before advanced models	A fancy model is justified only if it beats a simple one.
Human review for low confidence	Wrong routing is costly.

16. Roadmap across the six milestones

Milestone	What we will deliver
M1 Problem definition (this document)	Problem brief, users, objective, metrics, feasibility, repository
M2 Data preparation	Synthetic corpus, data card, exploratory analysis, cleaning, splits, privacy notes
M3 Baseline and model development	Baselines for all stages, stronger candidates, Git history, experiment log
M4 Evaluation and optimization	Evaluation report, error analysis, improvement rounds, robustness tests (including OCR)
M5 Deployment prototype	Working API or app, performance measurements, monitoring plan, failure handling
M6 Final delivery	Technical report, GitHub repository, live demo, business presentation, viva

17. M1 checklist

Required by the handbook	Where it is in this document
Problem brief	Sections 1 to 4
User identification	Section 5

Required by the handbook	Where it is in this document
Business objective	Section 6
Success metrics	Section 7
Feasibility analysis	Sections 8 to 13

18. Glossary

- **PII:** personally identifiable information, such as a name or SSN.
- **TPA:** third-party administrator, a company that handles claims for an insurer.
- **NER:** named entity recognition, finding items like names and dates in text.
- **OCR:** optical character recognition, turning a picture of text into text.
- **Synthetic data:** artificial data created by software, not taken from real people.
- **Baseline:** a simple first model used as the bar to beat.
- **Confidence score:** how sure the model is about its answer.
- **Calibration:** whether confidence scores match how often the model is right.
- **Human in the loop:** a person checks uncertain cases.
- **Model registry:** a store that keeps every model version with its details.